

IA-64: Rotating Registers

We noticed during the lab session and on the Ed forum that rotating registers were not understood well. This document aims to provide an additional support for the second part of the homework, specifically with the `loop.pip` instruction.

1 Definitions

Let's first clarify some key definitions of software pipelining, in the context of the IA-64 architecture.

1.1 Bundle

A bundle is a set of instructions that are fetched and executed at the same time. Each instruction within the bundle is assigned to execute on a distinct and predetermined execution unit.

If an execution unit is not utilized during a specific cycle, a nop instruction must be placed in the corresponding bundle slot.

*II = stage width in time
= the number of issue cycles / bundles between two consecutive loop-iteration starts
= the number of bundles in one software-pipeline stage
Number of stages = software-pipeline depth
= how many II-bundle stage groups are needed to cover the scheduled loop body*

1.2 Initiation Interval (II)

The *II* corresponds to the number of cycles between the start of successive iterations in a loop. Because a new loop iteration, composed of *II* bundles, enters the pipeline every *II* cycles, this metric acts as a way of measuring the throughput of the pipelined loop.

1.3 Stage

*In a VLIW processor, one issue cycle corresponds to issuing one instruction bundle.
Therefore, an initiation interval of 'II' cycles corresponds to 'II' bundles.
As a result, one software-pipeline stage is defined as a group of 'II' consecutive bundles.*

A loop stage is defined as a contiguous block of *II* bundles. The original loop body is divided into multiple stages, where each stage corresponds to a specific group of consecutive instructions belonging to a single logical iteration of the original loop.

During the execution of the loop, different stages from overlapping iterations are executed in parallel. To achieve this parallel execution, the final compiled kernel is compacted into a single block of *II* bundles, which represents the union of all corresponding bundles from overlapping stages.

The final bundle of this compacted block must contain the `loop.pip` instruction, which triggers the rotation of registers, update stage predicates and branch back to the beginning of the block.

1.4 Rotating Registers

Rotating registers are a specialized hardware feature that provides an automatic register renaming mechanism to prevent false dependencies between overlapping loop iterations.

Each time a loop instruction is executed (i.e., every *II* bundles), the processor's rotating register window is dynamically shifted by incrementing a hardware pointer known as the Register Rotation Base (RRB). To illustrate, if a value is produced in register `x32`, this mechanism implies that after an execution of the instruction `loop.pip`, the same value can be consumed in register `x33`.

2 Register Allocation

To determine exactly which physical rotating register a consumer instruction (i.e., that reads the register from the producer) should read from, the compiler applies the formula $xZ(+a, +b) = x(Z + a + b)$, where:

- *Z* is the base rotating register allocated to the producer (e.g., starting at `x32`).

- a is the iteration offset (+1 if produced in the preceding iteration; 0 if produced in any stage of the present iteration).
- b is the stage offset (the difference between the stage of the consumer instruction and the stage of the producer instruction). Because stages are 0-indexed, its maximum value is $\#stages - 1$.

Using this logic, we find that the longest lifetime of a rotating register is defined as:

$$\text{MaxLifetime} = Z + 1 + (\#stages - 1) = Z + \#stages$$

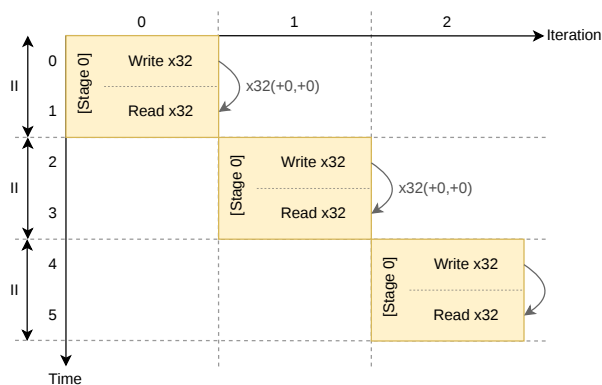
This implies that for every instruction that produces a value (i.e., a producer) and requires a destination register within the loop, the compiler allocates one rotating register every $\#stages + 1$ registers.

When performing register allocation, the procedure is to first focus exclusively on the loop body. Once the loop body is successfully allocated, the compiler then handles the prologue and epilogue. This two-step approach ensures that the most common operation (i.e., the loop body) is optimized and functioning correctly before adapting the two other parts.

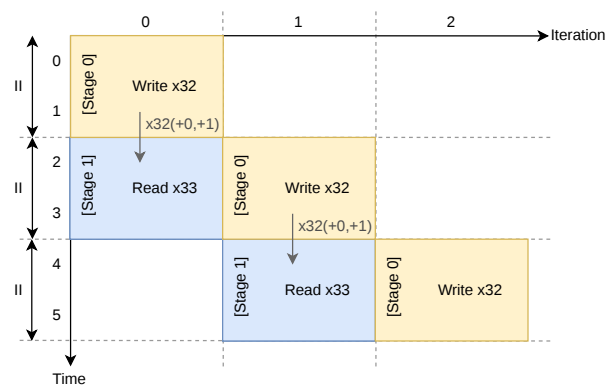
2.1 Simple Example

Consider now the following Figure 1, that represents the execution of a software pipelined loop with an II of 2 and 2 stages for the loop body.

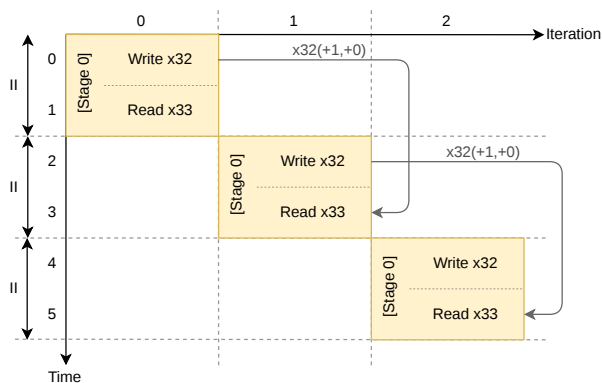
At every time step, a stage of the loop is executed and registers rotate at the end of their execution. Every column corresponds to a single loop iteration, where first stage 0 executes, before stage 1, etc.



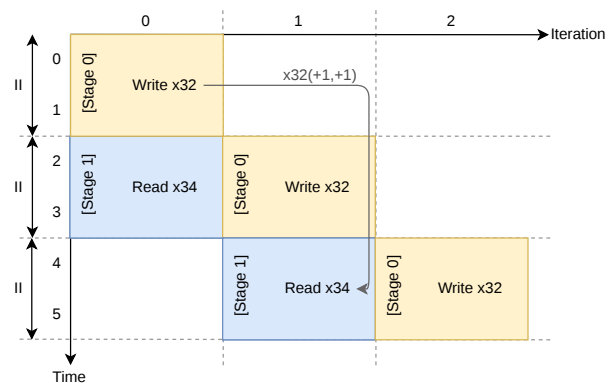
(a) Read from the same iteration, same stage, at $x32(+0,+0)$



(b) Read from the same iteration, next stage, at $x32(+0,+1)$



(c) Read from the next iteration, same stage, at $x32(+1,+0)$



(d) Read from the next iteration, next stage, at $x32(+1,+1)$

Figure 1: All possible cases of reading a register with $II = 2$, 2-stage loop body.

Suppose an instruction writes a value to the base register $x32$ in stage 0. In this case, four dependency cases are possible:

- (a) $(+0, +0)$, the consumer instruction is located in the exact same cell in the grid as the producer. Because they execute within a single stage, the `loop.pip` instruction has not been executed yet and no rotation occurred.
- (b) $(+0, +1)$, the consumer belongs to the same iteration, but executes in the next stage as it could not be scheduled earlier due to a RAW dependency or a structural hazard. Because a `loop.pip` instruction has been executed, the registers have rotated exactly once, meaning the consumer must read from `x33`.
- (c) $(+1, +0)$, the consumer belongs to the next iteration but the same stage, reading a value generated in the previous iteration. Since registers have again been rotated exactly once, the consumer must read from `x33`.
- (d) $(+1, +1)$, the consumer belongs to the next iteration and the next stage. The `loop.pip` instruction has been executed twice, meaning registers have been also rotated twice.